

CSA5801

DEVSECOPS ENGINEERING AND APPLICATION SECURITY

CO4 AT

TECHNICAL REPORT

IAM Relationship Modelling and GitHub Integration Using RStudio

STUDENT	Abinesh M
REGISTER NUMBER	192465050
COURSE	CSA5801 - DevSecOps Engineering and Application Security
REPORT SCOPE	Documentation of IAM modelling, RBAC simulation, RStudio/DiagrammeR implementation and GitHub workflow

01 INTRODUCTION

Identity and Access Management (IAM) is a central security mechanism for controlling who can access a system, what role that identity holds, and which actions are permitted. In a DevSecOps environment, these controls become especially important because software development involves shared repositories, source code, branches, pull requests and administrative resources. The practical titled “**IAM Relationship Modelling and GitHub Integration Using RStudio**” applies these ideas through a small, reproducible R-based model.

The practical objective is to model IAM relationships, simulate Role-Based Access Control (RBAC), test authorized and unauthorized access decisions, and upload the RStudio project to GitHub independently. The specified software stack is RStudio, R, Git and a GitHub account. The practical also states that an organization account, enterprise identity provider or administrator assistance is not required for the exercise.

The implementation therefore follows a compact security chain: **User** → **Role** → **Permission** → **Access Decision** → **GitHub Resource**. The value of the experiment is not the size of the codebase, but the explicit demonstration that permissions can be represented as rules, evaluated consistently, visualized as relationships and placed under version control.

Report approach. The first half establishes the IAM/RBAC model and explains the implementation. The second half records the observed outputs, interprets the access decisions and documents the Git/GitHub workflow. The final section maps the work back to DevSecOps security principles and the practical learning outcome.

02 OBJECTIVE AND TECHNICAL SCOPE

The practical defines four closely connected activities: (1) model IAM relationships, (2) simulate RBAC access decisions, (3) test both authorized and unauthorized actions, and (4) maintain the RStudio project through Git and GitHub.

Area	What was implemented	Security relevance
IAM modelling	User, role, repository, source code, pull request and permission relationships	Relationships explicit
RBAC	Developer, Reviewer and Administrator role rules	Restricts actions according to assigned role
Decision testing	Five access requests evaluated as granted or denied	Demonstrates authorization enforcement
Visualization	DiagrammeR graph generated from R	Communicates IAM relationships visually
Version control	RStudio project prepared for Git	Tracks and maintains the practical code
GitHub integration	Repository created and R program pushed	Places the practical under collaborative version control

Defined project identifiers: GitHub repository: **devsecops-iam-lab**; RStudio project: **DevSecOps_IAM**. These identifiers are explicitly specified by the practical.

03 IAM AND RBAC MODEL

The experiment uses RBAC as the decision model. Instead of assigning individual permissions independently to every user, a user is associated with a role and the role determines the allowed operations. This creates a clean separation between identity and authorization policy.

Core model: User → Role → Permission → Requested Action → Policy Evaluation → Access Decision.

Identity	Role	Permission	Representative action
Student_A	Developer	Write	Commit / Push / Create Branch
Student_B	Reviewer	Review	Review / Approve Pull Request
Faculty	Administrator	Admin	Repository administration

The practical explicitly defines this three-row access table, including Student_A as Developer with Write permission, Student_B as Reviewer with Review permission, and Faculty as Administrator with Admin permission.

This structure demonstrates the principle of **least privilege** at a simple laboratory scale: a Reviewer can review or approve a pull request, but is not given push access; a Developer can commit, push and create a branch, but cannot delete the repository; an Administrator is allowed to perform administrative operations.

04 IAM RELATIONSHIP VISUALISATION

The relationship model was implemented using the **DiagrammeR** package in R. The practical instructs the student to install/load DiagrammeR and use **grViz()** to render the IAM graph. The graph connects users to roles and roles to GitHub resources such as

source code, pull requests, the repository and permissions

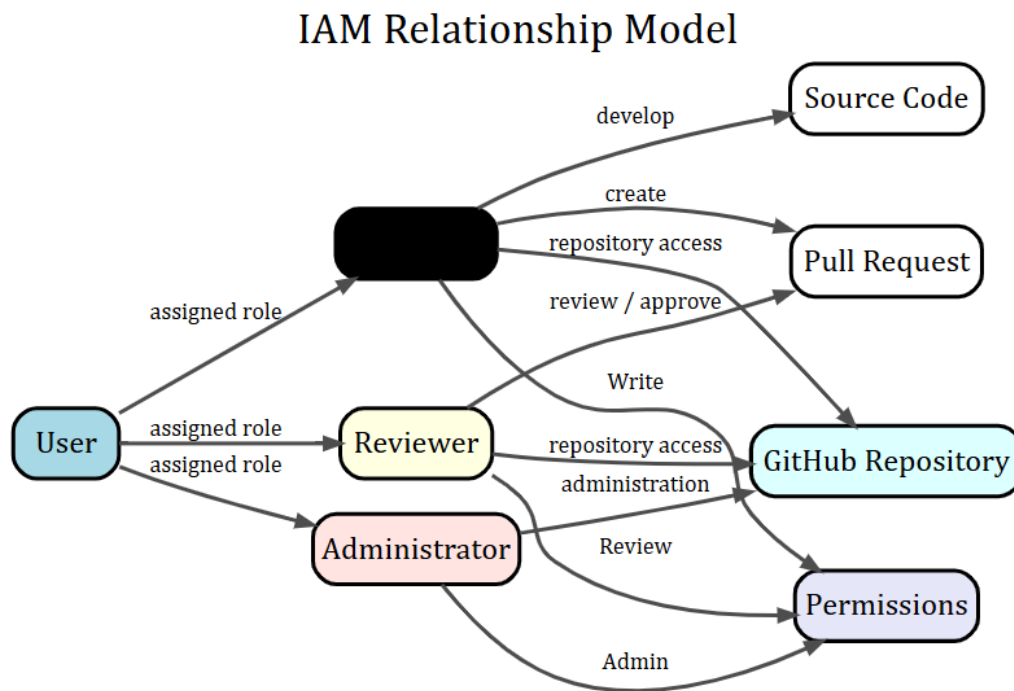


Figure 1. Captured DiagrammeR output showing the IAM relationship model used in the practical.

The model communicates several relationships at once: a User is assigned roles; a Developer is associated with source-code development, pull-request creation and repository access; a Reviewer is associated with review/approval activities; and an Administrator is associated with repository administration. The graph therefore acts as a visual summary of the authorization model rather than merely a decorative diagram.

Interpretation: the diagram separates the *who* (User), *what identity context* (Role), *what can be done* (Permission) and *where the action applies* (GitHub resource). That separation is the key IAM concept being demonstrated.

05 IMPLEMENTATION IN RSTUDIO

The R implementation was organized around a small number of reproducible components: package loading, relationship visualization, role/permission data structures, an access-control function and test execution. This makes the experiment easy to rerun and keeps the authorization logic visible in code.

The core access-control function evaluates the supplied role and requested action. An Administrator request is granted; Developer requests are granted only for Commit, Push and Create Branch; Reviewer requests are granted only for Review Pull Request and Approve Pull Request. Any other combination returns ACCESS DENIED. This logic follows the practical's supplied function.

Representative implementation logic:

```
check_access(role, action)
if role = Administrator → ACCESS GRANTED
if role = Developer and action ∈ {Commit, Push, Create Branch} → ACCESS GRANTED
if role = Reviewer and action ∈ {Review Pull Request, Approve Pull Request} → ACCESS GRANTED
otherwise → ACCESS DENIED
```

The observed RStudio output also records the role permission vectors. The Developer role contains Commit, Push and Create Branch; the Reviewer role contains Review Pull Request and Approve Pull Request; and the Administrator role contains both development/review operations and repository-management actions.

```
$Developer
[1] "Commit"          "Push"          "Create Branch"

$Reviewer
[1] "Review Pull Request" "Approve Pull Request"

$Administrator
[1] "Commit"          "Push"          "Create Branch"
[4] "Review Pull Request" "Approve Pull Request" "Delete Repository"
[7] "Manage Repository" "Manage Access"
```

Figure 2. Captured RStudio console output showing the role-permission definitions.

06 ACCESS-CONTROL SIMULATION

The practical requires five explicit access tests. These tests are intentionally mixed: some represent actions that match a role's permissions, while others represent actions outside that role's authorization boundary. The expected sequence is GRANTED, DENIED, GRANTED, DENIED and GRANTED

Test	Role	Requested action	Observed decision	Interpretation
T01	Developer	Commit	ACCESS GRANTED	Within Developer permissions
T02	Developer	Delete Repository	ACCESS DENIED	Administrative action not granted to Developer
T03	Reviewer	Approve Pull Request	ACCESS GRANTED	Within Reviewer permissions
T04	Reviewer	Push	ACCESS DENIED	Push is not a Reviewer permission
T05	Administrator	Delete Repository	ACCESS GRANTED	Administrative operation

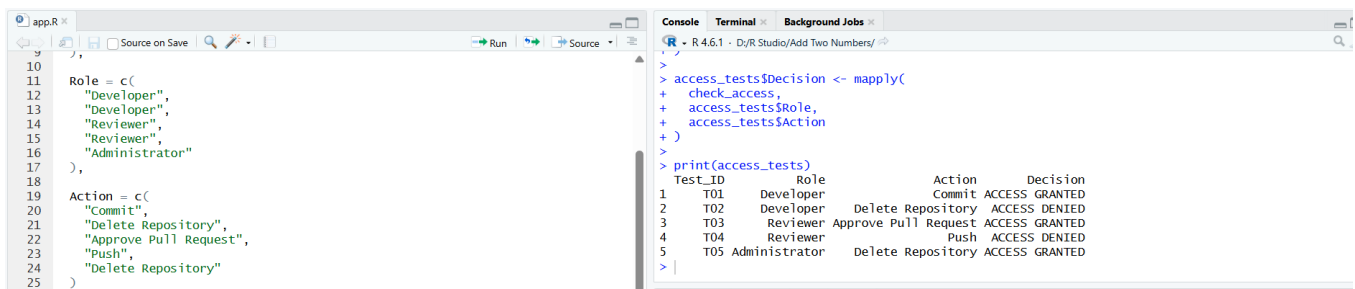


Figure 3. Captured RStudio output showing the five automated access decisions.

The results show **3 granted requests and 2 denied requests**. The denied cases are particularly important because they demonstrate that the model is not simply returning GRANTED for every request. Developer and Reviewer boundaries are being enforced according to the role-specific policy.

07 ACCESS MAPPING AND WORKFLOW

The IAM workflow implemented in the practical can be represented as a policy decision pipeline. A user enters the system with an assigned role. A requested action is evaluated against the role's permitted operations. The resulting decision determines whether the corresponding GitHub resource can be accessed or modified.

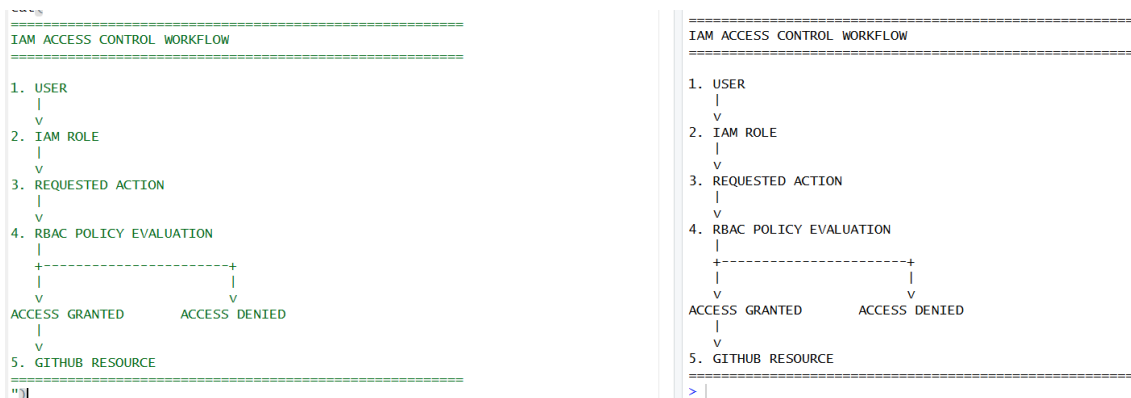


Figure 4. Captured console representation of the IAM access-control workflow.

Workflow interpretation:

Stage	Meaning in the experiment
1. User	The identity requesting access.
2. IAM Role	Developer, Reviewer or Administrator context.
3. Requested Action	Operation such as Commit, Push, Review or Delete Repository.
4. RBAC Policy Evaluation	Role/action combination is checked against the rules.
5. Access Decision	The request becomes ACCESS GRANTED or ACCESS DENIED.
6. GitHub Resource	The authorized operation applies to the repository-related resource.

```

      User          Role Permission
1 Student_A      Developer      Write
2 Student_B      Reviewer       Review
3 Faculty Administrator      Admin
>

```

Figure 5. Captured IAM user-role-permission table from RStudio.

08 GIT AND GITHUB INTEGRATION

After the IAM model and access tests were implemented, the practical moves the RStudio project into a Git-controlled workflow. The specified repository is **devsecops-iam-lab**, while the RStudio project is named **DevSecOps_IAM**.

The Git configuration step requires checking whether Git is detected under **Tools** → **Global Options** → **Git/SVN**, followed by selecting Git under the project's version-control settings. The R program is then committed from the RStudio Git pane using the message **"IAM relationship modelling completed"**

The final push transfers the committed R program to GitHub. The practical asks the student to verify that **iam_github.R** appears in the repository after pushing

Repository structure defined by the practical:

```

devsecops-iam-lab/
■■■ iam_github.R
■■■ README.md

```

This integration is important from a DevSecOps perspective because the security experiment itself becomes version-controlled engineering work. The policy logic, visualization code and access tests are no longer isolated in a local RStudio session; they can be maintained, reviewed and reproduced through Git/GitHub.

09 RESULTS, SECURITY ANALYSIS AND OBSERVATIONS

The implementation demonstrates a small but complete authorization loop. The user-role mapping establishes the security context; the permission matrix establishes allowed actions; the access function performs the policy decision; and the test cases verify that both valid and invalid requests produce the expected outcome.

Observation	Security meaning
Developer can Commit	Development work is permitted under the Developer role.
Developer cannot Delete Repository	Repository administration is separated from ordinary development.
Reviewer can Approve Pull Request	Review responsibility is explicitly assigned.
Reviewer cannot Push	Review and code modification responsibilities remain separated.
Administrator can Delete Repository	Administrative authority is represented by a distinct role.

Security principles demonstrated

Principle	How the practical demonstrates it
Role-based authorization	Actions are evaluated against a role rather than treated as universally available.
Least privilege	Developer and Reviewer roles receive narrower capabilities than Administrator.
Separation of duties	Review/approval is separated from ordinary push activity.
Explicit authorization	Every tested action produces an explicit granted/denied result.

Principle	How the practical demonstrates it
Traceability through version control	The R program is prepared for Git commit and GitHub storage.

The practical's expected learning outcome is to enable students to model IAM relationships using R, implement basic RBAC rules, test authorized and unauthorized actions, use Git from RStudio and maintain a DevSecOps laboratory project in GitHub. The completed work directly addresses those outcomes. ■filecite■turn0file0■L140-L143■

10 EVIDENCE AND SUBMISSION MAPPING

The practical specifies five submission components: the R program, IAM relationship diagram screenshot, access-control output screenshot, GitHub repository link and a screenshot showing successful push.

Required evidence	Status in this report	Where represented
R program (iam_github.R)	Implemented	Implementation described; source file submitted separately
IAM relationship diagram	Included	Figure 1
Access-control output	Included	Figure 3 and access table
GitHub repository link	Submission artifact	Repository maintained separately
Successful push screenshot	Submission artifact	Attach the final Git/GitHub screenshot if required

Important evidence note: The screenshots supplied for this report document the R/DiagrammeR and RBAC stages. The practical separately requires the GitHub repository link and successful-push screenshot; these should remain attached to the final submission even if they are not reproduced in this report.

The report therefore functions as the explanatory documentation layer, while the R file and GitHub repository remain the primary implementation artifacts. This separation keeps the report readable and avoids replacing executable evidence with decorative screenshots.

11 CONCLUSION

The CO4 practical demonstrates how a simple IAM policy can be modelled, visualized, executed and version-controlled within a DevSecOps workflow. Using R and DiagrammeR, the relationships among users, roles, permissions and GitHub resources were represented as a graph. Using R functions, the same model was converted into executable access-control decisions.

The five required tests produced three authorized decisions and two denied decisions. The denied cases are meaningful because they verify that role boundaries are active: a Developer is not allowed to delete the repository and a Reviewer is not allowed to push changes. The Administrator role retains the broader administrative authority defined by the practical.

The Git integration stage extends the experiment beyond a local demonstration. By preparing **iam_github.R** for commit and push to **devsecops-iam-lab**, the practical connects security policy modelling with a real version-control workflow. This reflects the DevSecOps idea that security controls should be implemented as repeatable, inspectable and maintainable engineering artifacts.

Overall, the completed work satisfies the practical's central learning goals: IAM relationship modelling, RBAC implementation, access-decision testing, Git usage in RStudio and GitHub-based project maintenance.

12 APPENDIX: PRACTICAL WORKFLOW AT A GLANCE

1. Create GitHub repository → devsecops-iam-lab
2. Create RStudio project → DevSecOps_IAM
3. Install/load DiagrammeR → generate IAM graph with grViz()
4. Create IAM table → Student_A / Student_B / Faculty
5. Implement RBAC → Developer / Reviewer / Administrator
6. Test access decisions → 5 required cases
7. Enable Git → RStudio Git/SVN settings
8. Commit → "IAM relationship modelling completed"
9. Push → GitHub repository
10. Verify → iam_github.R present in repository

Final conceptual chain: USER → ROLE → PERMISSION → REQUEST → RBAC POLICY → ACCESS DECISION → GITHUB RESOURCE

Source basis: CSA58 practical document, *IAM Relationship Modelling and GitHub Integration Using RStudio*, 3 pages. This report intentionally does not introduce an independent Part 3 experiment because the supplied practical does not specify one.